04 Kontrollstrukturen

Kontrollstrukturen

Vorlesung 22.10.2026
Quelle: 04_Kontrollstrukturen.pdf

if-Anweisung

Vorlesung 22.10.2026 – „Kontrollstrukturen“
Themen: if-Anweisungen · Schleifen (`while`, `do-while`, `for`, `foreach`) · Switch-Statement und Switch-Expression

if-then-else

```
// Normale Version
if (Bedingung) {
    Anweisungsliste
} else {
    Anweisungsliste
}
…

// Alternativ ohne else-Fall
if (Bedingung) {
    Anweisungsliste
}
…
```

Ablauf
1. Prüfe die Bedingung.
2. `true` → führe den **if-Block** aus, fahre danach im Programm fort.
3. `false` → führe den **else-Block** aus (falls vorhanden), fahre danach im Programm fort.

Varianten

Geschachtelte if-Anweisung	if-Kaskade
`if` in einem `if`-Block; `else` jeweils optional	Folge von `else if`

```
// Geschachtelt
if (Bedingung1) {
    Anweisungsliste
    if (Bedingung2) {
```

```
        Anweisungsliste
    } // else optional
    else {
        Anweisungsliste
    }
} // else optional
else {
    Anweisungsliste
}

// if-Kaskade
if (Bedingung1) {
    Anweisungsliste
} else if (Bedingung2) {
    Anweisungsliste
} else if (Bedingung3) {
    Anweisungsliste
} …
else {
    Anweisungsliste
}
```

Beispiel Schaltjahr

Regel: Ein Jahr ist ein Schaltjahr, wenn es durch **4** teilbar ist. Ist es auch durch **100** teilbar, dann nur, wenn es zusätzlich durch **400** teilbar ist.

Lösung der Folie (Übung 1: **eine** Bedingung; Übung 2: Teilbedingungen in eigenen `if`s):

```
public static void main(String[] args) {
    int year = Integer.parseInt(args[0]);

    // Übung 1
    if (year % 4  0 && year % 100 != 0 || year % 400  0) {
        System.out.println("Schaltjahr1");
    }

    // Übung 2
    if (year % 4 == 0) {
        if (year % 100 != 0) {
            System.out.println("Schaltjahr2");
        }
    }
    if (year % 400 == 0) {
        System.out.println("Schaltjahr3");
    } else {
        System.out.println("kein Schaltjahr3");
    }
}
```

Priorität
In Übung 1 bindet `&&` stärker als `||` (03 Kontrollfluss und Bedingungen), also `(year % 4  0`

```
    && year % 100 != 0) || year % 400  0.
```

Bedingter Operator (ternärer Operator)

Entspricht einer „**if-Anweisung in einer Zeile**“:

```
variable = Bedingung ? Ausdruck1 : Ausdruck2;
```

```
int value1 = …;
int value2 = …;
int result = value2 > 1 ? value1 / value2 : value1;
```

Äquivalent zu:

```
if (Bedingung) {
    variable = Ausdruck1;
} else {
    variable = Ausdruck2;
}
```

Teilweise Auswertung: nur zwei der drei Operanden werden berechnet, einer wird ignoriert (im Beispiel schützt `value2 > 1` vor der Division durch 0).

Schleifen

Schleifen sind Kontrollstrukturen zur **wiederholten Ausführung** von Anweisungen. Die Anzahl der Durchläufe wird über eine **Bedingung** gesteuert (03 Kontrollfluss und Bedingungen).

Schleife	Prüfung	Typ
`while`	**vor** dem Durchlauf	**abweisende** Schleife
`do-while`	**nach** dem Durchlauf (min. 1×)	**annehmende** Schleife
`for`	vor dem Durchlauf, mit **Schleifenzähler**	bekannter Wertebereich

while-Schleife

```
while (Bedingung) {
    Anweisungsliste
    Anpassung der Bedingung
}
…
```

Ablauf

1. Prüfe die Bedingung.
2. `true` → führe den `while`-Block aus, gehe zurück zu 1.
3. `false` → verlasse die Schleife, fahre im Programm fort.

> **Endlosschleife**
> Die Bedingung muss im Schleifenkörper **angepasst** werden, sonst droht eine Endlosschleife.

Übung: alle Vielfachen von drei zwischen 1 und `boundary` ausgeben (Lösung der Folie):

```
private static final int DIV = 3;
```

```java
public static void main(String[] args) {
    int boundary = Integer.parseInt(args[0]);
    // ggf. Prüfung von boundary.
    int counter = 1;
    while (counter <= boundary) {
        if (counter % DIV == 0) {
            System.out.println(counter);
        }
        counter++;
    }
}
```

do-while-Schleife

Variante der `while`-Schleife: die Abbruchbedingung wird **erst nach dem ersten Durchlauf** geprüft.

```
do {
    Anweisungsliste
    Anpassung der Bedingung
} while (Bedingung)
…
```

Ablauf: 1. Gehe in den `do`-Block → 2. führe Anweisungen aus → 3. prüfe Bedingung: `true` → zurück zu 1; `false` → Schleife verlassen.

> **Der `do`-Block läuft** mindestens einmal**.**
> `do-while` = **annehmend**, `while` = **abweisend**.

Übung: alle Zahlen von `boundary` abwärts bis zum größten Vielfachen einer Zahl `value` (mit `value <= boundary`) ausgeben (Lösung der Folie):

```java
public static void main(String[] args) {
    int boundary = Integer.parseInt(args[0]);
    int value = Integer.parseInt(args[1]);
    do {
        System.out.println(boundary);
        boundary = boundary - 1;
    } while (boundary % value != 0);
}
```

Beispiel `boundary = 10`, `value = 3`: gibt `10` aus, danach ist `boundary = 9` (Vielfaches von 3) → Abbruch. Das Vielfache selbst (9) wird in dieser Folienlösung **nicht** mehr ausgegeben; das ist ein Effekt der `do-while`-Reihenfolge (erst ausgeben, dann prüfen).

for-Schleife

Wird verwendet, wenn ein **bekannter Wertebereich** mit einem **Schleifenzähler** durchlaufen werden soll (z.B. Zahlen 1 bis 100, Array bestimmter Länge).

```
for (Initialisierung sz; Bedingung; sz weiterzählen) {
    Anweisungsliste
}
…
```

Ablauf
1. Initialisiere den Schleifenzähler.
2. Prüfe die Bedingung: `true` → Anweisungsliste ausführen, Zähler anpassen, zurück zu 2; `false` → Schleife verlassen.

Beispiel/Übungen (Lösungen der Folien):

```
// Übung 1: Vielfache von 3 bis boundary
int boundary = Integer.parseInt(args[0]);
for (int i = 0; i < boundary; i = i + 3) {
    System.out.println(i);
}

// Übung 2: Quersumme einer positiven Ganzzahl
int value = Integer.parseInt(args[0]);
int sum = 0;
for (int i = value; i > 0; i = i / 10) {
    int next = i % 10;      // letzte Ziffer
    sum = sum + next;
}
System.out.println("Die Quersumme von " + value + " ist: " + sum);
```

> **Anmerkung zur Folienlösung (Übung 1)**
> Die Lösung startet bei `i = 0` und prüft `i < boundary`: sie gibt also auch die `0` aus und lässt `boundary` selbst aus, wenn es ein Vielfaches von 3 ist. Für „zwischen 1 und boundary“ wäre `for (int i = 3; i <= boundary; i += 3)` genauer.

> **Quersumme**
> `i % 10` liefert die letzte Ziffer, `i / 10` (ganzzahlige Division, 02 Variablen und Datentypen) schneidet sie ab.

Operatoren zur Schleifenkontrolle

Operatorzuweisung (mit allen arithmetischen Operatoren möglich):

```
int i = 1;
i += 2;   // entspricht i = i + 2;
i -= 2;   // entspricht i = i - 2;
```

Inkrement `++` (nur Hochzählen in Einserschritten) und **Dekrement** `--` (nur Herunterzählen in Einserschritten):

```
int i = 1;  i++;   // i = i + 1
int i = 10; i--;   // i = i - 1
```

Präfix- und Postfix-Schreibweise

	Präfix	Postfix
Inkrement	`++variable`	`variable++`
Dekrement	`--variable`	`variable--`
Wirkung	**erst** ändern, **dann** Wert liefern	**erst** Wert liefern, **dann** ändern

Aufgabe: Ausgabe der vier Beispiele von der Folie →

foreach

In den Themen der Vorlesung angekündigt (`while`, `do-while`, `for`, `foreach`), auf den Folien aber **nicht ausgeführt**. Hier fehlt also noch Stoff (Platz für eigene Notizen, Bezug: 02 Variablen und Datentypen).

Switch

Switch-Statement

Ähnelt entfernt einer einfachen if-Kaskade: abhängig vom Ausdruck (*expression*) werden die Anweisungen zu einem bestimmten Fall (*case*) ausgeführt.

```
switch (expression) {
    case label1:
        Anweisungsliste
        break;
    case label2:
        Anweisungsliste
        break;
    case label3:
        …
    default:
        Anweisungsliste
        break;
}
…
```

Ablauf

1. Werte `expression` zu einem Wert aus.
2. Vergleiche den Wert mit dem Label des nächsten `case`.
3. Gleich → führe alle Anweisungen im `case` aus, **bis `break`** erreicht wird; fahre danach im Programm fort.
4. Nicht gleich → weiter zum nächsten `case` (zurück zu 3).
5. Passt kein `case`: prüfe, ob es einen `default`-Fall gibt.
6. Falls ja: führe dessen Anweisungen bis `break` aus.
7. Fahre im Programm fort.

Was ist zu beachten?

- `expression` muss zu `int`, `String`, `char` oder `Enum` ausgewertet werden.
- `case`-Labels müssen innerhalb eines `switch` **eindeutig** sein (keine doppelten Werte).
- `case`-Labels müssen **konstante Werte** sein (Zahl, Buchstabe, String).
- `case`-Labels können **zusammengefasst** werden. **Fall-through**: es werden alle `case`s bis zum ersten `break` ausgeführt.
- `default`: Standardbehandlung, **optional**, darf nur **einmal am Ende** stehen, wird durchlaufen, wenn kein anderer `case` gilt.

Beispiel: ausführlich vs. kompakt

```
// ausführlich (ohne Fall-through)
switch (numberOfDay) {
    case 1: System.out.println("Werktag"); break;
    case 2: System.out.println("Werktag"); break;
```

```
    // … 3, 4, 5 genauso
    case 0: System.out.println("Wochenende"); break;
    case 6: System.out.println("Wochenende"); break;
    default: System.out.println("kein gültiger Wochentag");
}

// kompakt (Fall-through sinnvoll genutzt)
switch (numberOfDay) {
    case 1, 2, 3, 4, 5:
        System.out.println("Werktag");
        break;
    case 0, 6:
        System.out.println("Wochenende");
        break;
    default:
        System.out.println("kein gültiger Wochentag");
}
```

Switch als Ausdruck mit Zuweisung zu einer Variable

Eine **Switch-Expression** liefert einen Wert, der einer Variable zugewiesen wird. **Strichpunkt nach } ist nötig** wegen der Zuweisung.

Pfeil-Schreibweise (State of the Art): **kein Fall-through, kein `break`** mehr notwendig.

```
String day1 = switch (numberOfDay) {
    case 1, 2, 3, 4, 5 -> "Werktag";
    case 0, 6 -> "Wochenende";
    default -> {
        // Pfeil erlaubt die Verwendung von Code-Blöcken.
        System.out.println("Fehler: kein gültiger Wochentag");
        yield "kein gültiger Wochentag";
    }
};  // Strichpunkt hier notwendig wegen Zuweisung
System.out.println(day1);
```

Doppelpunkt-Schreibweise mit `yield` (vergleichbar zum Pfeil, **keine Code-Blöcke** möglich):

```
String day2 = switch (numberOfDay) {
    case 1, 2, 3, 4, 5:
        yield "Werktag";
    case 0, 6:
        yield "Wochenende";
    default:
        yield "kein gültiger Wochentag";
};  // Strichpunkt hier notwendig wegen Zuweisung
System.out.println(day2);
```

`yield` liefert den Wert des Zweigs als Ergebnis des `switch`-Ausdrucks.

Übung Tage pro Monat

Klasse `SwitchExcercise` mit `main`. Switch-Ausdruck, der für jeden Monat die Anzahl der Tage (31, 30, 28 oder 29 bei Schaltjahr) liefert. Eingabe: Monatskürzel (`JAN`, `FEB`, …) und Jahr; Pfeil-Schreibweise. Lösung der Folie:

```
static void main(String[] args) {
    String month = args[0];
    int year = Integer.parseInt(args[1]);

    int numberOfDays = switch (month) {
        case "JAN", "MAR", "MAI", "JUL", "AUG", "OKT", "DEZ" -> 31;
        case "FEB" -> {
            if (year % 4  0 && year % 100 != 0 || year % 400  0) {
                yield 29;
            }
            yield 28;
        }
        default -> 30;
    };
    System.out.println(numberOfDays);
}
```

Schaltjahr-Bedingung:

Zusammenfassung der Vorlesung 4

- Ein Programm besteht aus **einfachen** und **zusammengesetzten** Anweisungen.
- Zusammengesetzte Anweisungen = **Kontrollstrukturen**, die den Kontrollfluss steuern.
- Zur Verfügung stehen einfache Anweisungen (Zuweisungen, Methodenaufrufe), bedingte Anweisungen und Schleifen; alle **beliebig tief verschachtelbar**.
- Bedingte Anweisungen und Schleifen werden über **Bedingungen** gesteuert (arithmetische, relationale, logische Operatoren).
- Methoden, bedingte Anweisungen und Schleifen definieren **Blöcke** mit geschweiften Klammern.

Übungsaufgaben

> **Lösungen sind eingeklappt (Pfeil anklicken). Wo die Folie keine Lösung zeigt, ist es meine eigene, bitte gegenprüfen.**

Übung Prä- und Postfix-Ausgaben

Wie sieht jeweils die Ausgabe aus? ()

```
// Beispiel 1
int i = 2;
while (i >= 0) {
    System.out.print(--i);
}
```

```
// Beispiel 2
int i = 2;
while (i >= 0) {
```

```
// Beispiel 3
int i = 2;
do {
    System.out.print(i);
} while (--i >= 0);
```

```
// Beispiel 4
int i = 2;
do {
```

```
    System.out.print(i--);              System.out.print(i);
}                                   } while (i-- >= 0);
```

Lösung (eigene Herleitung)

1. `10-1` (`--i` ändert zuerst: gibt 1, 0, −1 aus; bei i = −1 endet die Schleife)
2. `210` (`i--` liefert zuerst den alten Wert: 2, 1, 0)
3. `210` (ausgeben, dann `--i >= 0` prüfen: 2, 1, 0, dann i = −1 → Ende)
4. `210-1` (`i-- >= 0` vergleicht den **alten** Wert, daher läuft die Schleife noch einmal mit i = −1)